

README / عماع قرطان

وSQLAlchemy، مع واجهة عربية RTL افتراضية ودعم كامل للأجهزة البيومترية (Hikvision و ZKTeco).
(Multi-Company / لإدارة حضور وانصراف الموظفين، مبني بلغة Python باستخدام Qt6 PySide6)
نظام سطح مكتب متعدد الشركات (Multi-Tenant)

Built for organizations that need to manage attendance across an unlimited number of independent companies from a single installation, with full data isolation between tenants, biometric device integration, and Arabic-first bilingual reporting.

المحتويات / Contents

- نظرة عامة / Overview
- الميزات الرئيسية / Key Features
- البنية المعمارية / Architecture
- حزمة التقنيات / Tech Stack
- هيكل المشروع / Project Structure
- التثبيت والتشغيل / Installation & Running
- متغيرات البيئة / Environment Variables
- أول تشغيل / First Run
- نظام الترخيص / Licensing System
- النسخ الاحتياطي / Backup & Restore
- الأجهزة البيومترية المدعومة / Supported Devices
- الاختبار / Testing
- الترخيص / License

نظرة عامة / Overview

إضافي) آليا من بصمات الأجهزة أو الإدخال اليدوي، وصولا إلى تقارير جاهزة للتصدير بصيغ Excel و PDF و CSV. حياة الحضور: من تسجيل الموظفين وأجهزة البصمة، مروراً بحساب الحضور اليومي (تأخير، انصراف مبكر، وقت يوفر النظام إدارة كاملة لدورة

شركة أخرى، رغم أنها تعمل جميعاً فوق قاعدة بيانات واحدة — إضافة شركة جديدة لا تتطلب أي تعديل في الكود. بالكامل عن غيرها: بياناتها، مستخدموها، أدوارها، أقسامها، موظفوها، أجهزتها، وإعداداتها لا تشارك ولا تتقاطع أبداً مع كل شركة مسجلة في النظام معزولة

The system covers the full attendance lifecycle: enrolling employees and biometric devices, automatically computing daily attendance (late arrivals, early leaves, overtime) from device punches or manual entry, and exporting ready-to-share reports in Excel, PDF, and CSV. Every company is fully data-isolated from every other, even though they all run on one shared database — onboarding a new tenant requires no code changes.

الميزات الرئيسية / Key Features

- خاصة بكل شركة. company_id: عزل كامل للبيانات بين الشركات عبر تعدد الشركات (Multi-Tenant) على مستوى طبقة المستودعات (Repository layer)، مع أدوار (Roles)

- معروضة) — كل من التطبيق المكتبي وواجهة REST يتشاركان نفس مصدر الصلاحية.الصلاحيات (RBAC) : تحقق فعلي من الصلاحيات وقت التشغيل على كل عملية وكل شاشة (وليس فقط قائمة صلاحيات
- — مع مخططات بيانية تنفيذية (اتجاه الحضور خلال 14 يوما، وتوزيع الموظفين حسب القسم).لوحة تحكم لحظية — الموظفون النشطون، الأقسام، حالة الأجهزة، وملخص حضور اليوم (حاضر/متأخر/غائب/إجازة) : إحصائيات
- فروع الشركة (المواقع الفعلية)، وتوليد رمز QR وباركود تلقائي لكل موظف.إدارة الموظفين والأقسام والفروع : بحث، تصنيف هرمي للأقسام (شجرة قابلة لإعادة الترتيب)، إدارة
- وطلبات إجازة قابلة للاعتماد — كلها تحتسب تلقائيا ضمن حالة الحضور اليومية.الورديات والعطلات والإجازات : تعريف ورديات العمل وتعيينها للموظفين، تقويم العطلات الرسمية،
- يدوي، واحتساب آلي لحالة اليوم (حاضر/متأخر/غياب/إجازة/عطلة) بالتوقيت المحلي للشركة.الحضور والانصراف : استيراد بصمات من الأجهزة، إدخال
- اتصال، مزامنة سجلات (يدويا أو تلقائيا حسب جدول)، ودفع بيانات الموظفين إلى الجهاز.الأجهزة البيومترية : دعم بروتوكولات (ZKTeco (TCP/UDP و(Hikvision (ISAPI، اختبار
- مزامنة تلقائية للأجهزة ونسخ احتياطي تلقائي للقاعدة، تعمل في الخلفية دون تدخل المستخدم (المهام المجدولة :SchedulerService).
- الوقت الإضافي، الغياب) بثلاث صيغ تصدير، مع دعم كامل للنص العربي RTL في ملفات PDF.التقارير : 6 أنواع تقارير (ملخص حضور، حسب الموظف، حسب القسم، المتأخرون،
- دقيقة (Permission catalog)، تدقيق كامل (Audit Log) لكل عملية حساسة.المستخدمون والصلاحيات : أدوار قابلة للتخصيص لكل شركة، صلاحيات
- صيغة التاريخ، العملة)، ونسخ احتياطي/استعادة آمن لقاعدة البيانات (متوافق مع وضع WAL، ومشفر).الإعدادات : ملف تعريف الشركة، تفضيلات العرض (المنطقة الزمنية،
- فاتحة/داكنة مستوحاة من Fluent Design DD/MM/YYYY.افتراضي، تنسيق تاريخ RTL :واجهة عربية أولا ، نظام 24 ساعة، أرقام عربية-هندية اختيارية، وسمة بصرية (Theme)
- قفل الحساب بعد محاولات فاشلة متكررة، انتهاء الجلسة عند عدم النشاط، وسجل تدقيق شامل.أمان : كلمات مرور مجزأة (bcrypt)، تشفير الحقول الحساسة في قاعدة البيانات (Fernet)،
- الشركات عبر HTTP لأغراض التكامل، معطلة افتراضيا.run_api.py: عملية منفصلة (واجهة REST اختيارية ، مبنية على FastAPI) تعرض نفس بيانات

البنية المعمارية / Architecture

وفصل صارم بين الطبقات:Repository Pattern مع Clean Architectureالنظام مبني وفق مبادئ

ui/	PySide6 windows/pages — Qt signals only	
controllers/	Session-per-operation, ORM → dict	
services/	Business logic, validation, audit trail	
repositories/	Data access — the tenant-isolation point	
models/	SQLAlchemy ORM (declarative, typed)	

↑ database/ (SQLAlchemy engine) ↓ devices/ (ZKTeco/Hikvision)

- عبر مستودع مفيد بشركة يضيف CompanyScopedRepository: العزل بين الشركات (Tenant Isolation) يمكن لخدمة أن "تنسى" هذا الشرط.... WHERE company_id = هو نقطة الإنفاذ الجيدة — أي استعلام تلقائيا، ولا

- إلى () session_scope(): كل عملية تنفذ داخل جلسة قاعدة بيانات مستقلة (نمط التحكم (Controllers) — لا تلمس الواجهة أبدا كائن SQLAlchemy قد يصبح منفصلا (dict.(Detached)، وتحول كائنات ORM قبل إغلاق الجلسة
- مشتركة في معرفات UUID عامة والقفل التفاضلي (Optimistic Locking) والحذف الناعم (Soft Delete) Mixins مطبقة عبر models/base.py.

Tech Stack / حزمة التقنيات

الطبقة	التقنية
الواجهة	PySide6 (Qt6)
قاعدة البيانات / ORM	SQLAlchemy 2.x (SQLite افتراضيا، PostgreSQL/MySQL DB_DIALECT)
الأمان	bcrypt، HMAC-SHA256
السجلات	Loguru
التقارير	openpyxl (Excel)، ReportLab (PDF)، CSV built-in
النص العربي في PDF	arabic-reshaper + python-bidi
رموز QR/الباركود	qrcode، python-barcode
الأجهزة البيومترية	pyzk (ZKTeco)، requests (Hikvision ISAPI)
الإعدادات	python-dotenv

Project Structure / هيكل المشروع

```
attendance_system/
├── main.py           # يبتكّم الـ Composition Root
├── run_api.py        # REST API
├── config.py         # إعدادات
├── requirements.txt
├── database/
│   └── database.py   # SQLAlchemy
├── alembic/          # migrations
├── models/           # ORM
├── repositories/     # Data Access
├── services/         # Business Logic
├── devices/          # ZKTeco/Hikvision
├── controllers/      # Qt, RBAC
├── ui/               # PySide6
│   ├── theme.py      # Theme
│   ├── widgets.py    # Widgets
│   ├── login_window.py
│   └── main_window.py
├── ...               # ...
├── api/              # FastAPI
├── utils/             # Utilities
├── tests/             # pytest
└── assets/           # Assets
```

Installation & Running / التثبيت والتشغيل

Prerequisites / المتطلبات

- Python 3.13+

- (اختياري) PostgreSQL أو MySQL إن أردت تجاوز SQLite الافتراضي

خطوات التثبيت / Setup

```
cd attendance_system
```

```
# Create a virtual environment
python3 -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Optionally configure via .env
cp .env.example .env          # Set environment variables for development

# Run the application
python main.py
```

تلقائياً بـ:main.py عند أول تشغيل، يقوم

- 1 (إلخ) ./logs/, data/إنشاء مجلدات التشغيل
- 2 تهيئة قاعدة البيانات وإنشاء جميع الجداول.
- 3 تهيئة قائمة الصلاحيات العامة (Permission catalog) إن لم تكن موجودة.
- 4 عرض شاشة تسجيل الدخول.

متغيرات البيئة / Environment Variables

هو مصدر الحقيقة الكامل).config.pyجميع المتغيرات اختيارية ولها قيم افتراضية معقولة للتطوير المحلي (

المتغير	الافتراضي	الوصف		
APP_ENVIRONMENT	production	development \	testing \	production
APP_SECRET_KEY	⚠ insecure placeholder	قيمة حقيقية - يجب تغييره في الإنتاج. يوقع رموز الجلسة وواجهة REST. ولد python -c "import secrets; print(secrets.token_urlsafe(48))"		
DB_DIALECT	sqlite	sqlite \	postgres \	mysql
DB_SQLITE_PATH	data/attendance.db	مسار ملف SQLite		
DB_HOST / DB_PORT / DB_USERNAME / DB_PASSWORD / DB_NAME	—	إعدادات PostgreSQL/MySQL		
SECURITY_BCRYPT_ROUNDS	12	تكلفة تجزئة كلمة المرور		
SECURITY_SESSION_TIMEOUT_MINUTES	30	مهلة الخمول قبل انتهاء الجلسة		
SECURITY_MAX_LOGIN_ATTEMPTS	5	محاولات الدخول قبل القفل		

المتغير	الافتراضي	الوصف		
SECURITY_LOGIN_LOCKOUT_MINUTES	15	مدة قفل الحساب		
UI_DEFAULT_THEME	light	light \	dark	
BACKUP_AUTO_ENABLED / BACKUP_INTERVAL_HOURS / BACKUP_RETENTION_COUNT	true / 24 / 14	النسخ الاحتياطي التلقائي (منفذة عبر SchedulerService) سياسة		
DEVICE_AUTO_SYNC_ENABLED / DEVICE_AUTO_SYNC_INTERVAL_MINUTES	true / 15	مزامنة الأجهزة التلقائية المجدولة		
DEVICE_ZKTECO_PORT / DEVICE_HIKVISION_PORT	4370 / 80	المنافذ الافتراضية للأجهزة		
LOG_LEVEL	INFO	مستوى التسجيل		
API_ENABLED	false	الاختيارية (عملية منفصلة، شغلها بـ python تفعيل واجهة REST run_api.py)		
API_HOST / API_PORT	127.0.0.1 / 8000	عنوان ومنفذ واجهة REST		
API_TOKEN_EXPIRES_MINUTES	480	صلاحية رمز الدخول لواجهة REST مدة		

أول تشغيل / First Run

ومستخدم مدير يمكن استخدام واجهة برمجية مباشرة (حتى تتوفر شاشة "تسجيل شركة جديدة" في نسخة لاحقة):
لا توجد شركة أو مستخدم افتراضي مجهز مسبقاً؛ يبدأ النظام بقائمة شركات فارغة. لإنشاء أول شركة

```
from database.database import get_database
get_database().initialize()

with get_database().session_scope() as session:
    from services.company_service import CompanyService
    from services.user_service import UserService
    from repositories.role_repository import RoleRepository
    from models.enums import UserRole

    company = CompanyService(session).create_company(name="■■■■ ■■■■■■■■")
    admin_role = next(
        r for r in RoleRepository(session, company_id=company.id).list_all()
        if r.code == UserRole.SYSTEM_ADMIN.value
    )
    UserService(session, company_id=company.id).create_user(
        username="admin",
        full_name="■■■■ ■■■■■■■■",
        password="ChangeMe123!",
        role_id=admin_role.id,
    )
```

يمكن تسجيل الدخول من التطبيق باسم المستخدم وكلمة المرور أعلاه، واختيار الشركة من قائمة تسجيل الدخول.
بعدها

نظام الترخيص / Licensing System

(Machine-Locked License)، مستقل تماما عن قاعدة البيانات وعن نظام تراخيص الشركات ضمن التطبيق (عند كل إقلاع للتطبيق، قبل شاشة تسجيل الدخول. models/license.py النظام محمي بترخيص مرتبط بالجهاز ، وهو خاص بحدود اشتراك كل شركة داخل النظام، لا بحماية التطبيق نفسه). يتحقق من الترخيص

The application is protected by a machine-locked license, fully independent of the database and of the in-app, per-company models/license.py (which governs each tenant's subscription limits, not the application's own protection). It is verified on every startup, before the login screen.

كيف يعمل / How it works

- والنظام والعنوان الفعلي للشبكة، تعرض في شاشة التفعيل لإرسالها إلى المورد. معرف الجهاز (Machine ID) بصمة SHA-256 مشتقة من اسم الجهاز
 - بالإنترنت إطلاقا. <signature>. <payload>. AMS1: نص موقع رقميا بخوارزمية Ed25519 (مفتاح الترخيص اختياريًا بجهاز محدد. يتحقق التطبيق من التوقيع باستخدام مفتاح عام مضمن في الكود فقط — لا يحتاج اتصالاً ،) يحدد نوع الترخيص (تجريبي/شهري/سنوي/دائم)، تاريخ الانتهاء، وربطاً
 - الترخيص المفعّل يخزن مشفراً (Fernet) بمفتاح مشتق من معرف الجهاز نفسه، في التخزين المحلي — نسخ هذا الملف إلى جهاز آخر ينتج ملفاً لا يمكن فك تشفيره هناك. data/license.dat:
 - يوما، تفعل ذاتيا من داخل التطبيق دون الحاجة لمفتاح من المورد، ومرة واحدة فقط لكل جهاز. النسخة التجريبية
- 14 :

إدارة الترخيص من داخل التطبيق / In-App License Management

تاريخ الانتهاء، معرف الجهاز، معرف الترخيص) مع خمسة أزرار لإدارة: الإعدادات → الترخيص بعد تسجيل الدخول، من تعديل على بقية تبويباتها): عرض تفصيلي لحالة الترخيص (اسم الشركة، اسم العميل، النوع، الحالة، تاريخ التفعيل، تبويب جديد ضمن شاشة الإعدادات الحالية، دون أي

- لصق مفتاح جديد يحل محل الحالي. تفعيل ترخيص
- فيها المنتهية) — يرفض مفاتيح التجديد من نوع تجريبي أو دائم، ويظهر تاريخ الانتهاء الجديد فوراً. تجديد الترخيص : مخصص للاشتراكات الشهرية والسنوية فقط (بما
- (أو أي طرف يملك المفتاح العام) دون الحاجة لأي مفتاح توقيع إضافي داخل التطبيق. تصدير طلب الترخيص مع تضمين مفتاح الترخيص الموقع الأصلي كاملاً، وهو ما يجعل الملف قابلاً للتحقق بشكل مستقل من قبل المورد : يحفظ ملف JSON يحتوي معرف الترخيص، اسم الشركة والعميل، معرف الجهاز، وتفاصيل التفعيل —
- الخطوة. يرسل العميل هذا الملف إلى المورد، الذي يصدر بعدها مفتاحاً جديداً لجهاز مختلف. إلغاء تفعيل الترخيص : يسمح الترخيص من هذا الجهاز فوراً، مع خيار حفظ ملف طلب النقل في نفس
- نسخ معرف الجهاز

البنية له مسبقاً (انظر أدناه) دون الحاجة لأي تعديل على واجهات الترخيص الحالية. ملاحظة مهمة حول نقل التراخيص (جديد) وليس ضماناً تشفيرياً فوراً — تحقيق ذلك بشكل آلي يتطلب خادم ترخيص مركزي، وهو بالضبط ما تم تجهيز تفعيل نفس الترخيص على جهازين في آن واحد هو إجراء تنظيمي (المورد يراجع ملف طلب النقل قبل إصدار مفتاح : بما أن النظام يعمل بالكامل دون اتصال بالإنترنت، فإن منع

Post-login, from Settings → License (a new tab on the existing Settings screen — no other tab was touched): a full status view plus five actions — Activate, Renew (Monthly/Yearly only, including already-expired ones; rejects Trial/Lifetime renewal keys and shows the new expiry immediately), Export License Request (a JSON file embedding the original signed key itself as independently-verifiable proof — no extra signing key needed in the app), Deactivate (clears this machine immediately, optionally exporting a transfer request in the same step for the vendor to review before issuing a replacement key elsewhere), and Copy Machine ID. Preventing simultaneous activation on two machines is, honestly, a procedural control here (vendor reviews the transfer request) rather than a real-time cryptographic guarantee — that

needs the online-backend extension point described below, with zero changes required to these screens when it's added.

مفاتيح الترخيص (للمورد فقط) / (Issuing License Keys (Vendor-Only / إصدار

أداة سطر أوامر منفصلة تماما عن التطبيق، لا تستدعى منه أبدا:

```
# [ملاحظة: يجب أن يكون المسار المسمى /secure/ موجودا في النظام]
python -m licensing.license_generator generate-keypair \
  --private-key-out /secure/location/private_key.pem \
  --public-key-out /tmp/public_key.pem
# [ملاحظة: يجب أن يكون المسار المسمى /secure/ موجودا في النظام]
python -m licensing.license_generator issue \
  --private-key /secure/location/private_key.pem \
  --customer "XXXX XXXXX" --company "XXXX XXXXX" \
  --type yearly \
  --machine-id <Machine-ID-XXXX-XXXX-XXXX>

# [ملاحظة: يجب أن يكون المسار المسمى /secure/ موجودا في النظام]
python -m licensing.license_generator issue \
  --private-key /secure/location/private_key.pem \
  --customer "XXXX XXXXX" \
  --type lifetime
```

رفعه إلى نظام التحكم بالإصدارات (مستثنى فعليا عبر licensing/vendor/private_key.pem :تحذير أمني هو المفتاح الخاص الذي يسمح بإصدار تراخيص صالحة لهذا التطبيق. لا يجوز مطلقا تضمينه في التطبيق الموزع أو المفتاح العام المضمن حاليا؛ تسريبه يعني إمكانية تزوير تراخيص من قبل أي طرف يحصل عليه.gitignore). فقدانها يعني عدم القدرة على إصدار تراخيص جديدة لنفس

ترخيص عبر الإنترنت لاحقا / Extending to an Online License Server / التوسع ل خادم

في LicensePayload -> verify(key) بسيطة (LicenseBackend طبقة التحقق مبنية خلف واجهة نفس الواجهة وتمريضه إلى LocalLicenseBackend. التنفيذ الحالي (licensing/license_service.py) اتصال بالإنترنت. لإضافة تحقق عبر خادم لاحقا (مثل التأكد من عدم إلغاء الاشتراك)، يكفي إنشاء صنف جديد يحقق على واجهة التفعيل أو التخزين المحلي أو بقية التطبيق. (LicenseService(backend=...) يعمل بالكامل دون — دون أي تعديل

النسخ الاحتياطي / Backup & Restore

فورية، استعراض النسخ السابقة، والاستعادة منها. الآلية تستخدم واجهة الإعدادات → النسخ الاحتياطي من شاشة — يضمن ذلك عدم فقدان بيانات لم تكتب بعد إلى القرص. (sqlite3.Connection.backup() :إنشاء نسخة : الاستعادة تستبدل جميع البيانات الحالية بشكل نهائي. تنبيه الأمانة مع وضع WAL، وليس نسخا خام للملف

الأجهزة البيومترية المدعومة / Supported Devices

البروتوكول	الاتصال	الحقل المطلوب
ZKTeco TCP/IP	المنفذ 4370 افتراضيا، TCP	كلمة اتصال الجهاز (اختياري)
ZKTeco UDP	المنفذ 4370 افتراضيا، UDP	كلمة اتصال الجهاز (اختياري)
Hikvision (ISAPI)	المنفذ 80 افتراضيا، HTTP Digest Auth	اسم_المستخدم:كلمة_المرور

مكررة عند إعادة المزامنة، (Employee Number) الرقم الوظيفي مطابقة سجلات الجهاز بالموظفين تتم عبر (ومزامنة السجلات آمنة عند التكرار (لا تنشئ بصمات

الاختبار / Testing

التحكم، وواجهة REST كاملة عبر `pytest.ini` (انظر `tests/` مجموعة اختبارات `pytest` حقيقية موجودة في محاكاة/mock لطبقة ORM)، وتغطي احتساب الحضور، صلاحيات RBAC، التشفير، الفروع، المهام المجدولة، لوحة (TestClient) — كل اختبار يشغل قاعدة بيانات SQLite معزولة وحقيقية خاصة به (لا

```
pip install -r requirements.txt # pytest pytest-qt httpx
pytest
```

تم التحقق يدويا من كل شاشة واجهة أثناء التطوير عبر سيناريوهات تشغيل حقيقية ضد حقيقي. لتشغيل التطبيق في بيئة بلا شاشة (headless) `QApplication` كما

```
QT_QPA_PLATFORM=offscreen python main.py
```

الترخيص / License

والتوزيع التجاري. `assets/fonts/DejaVuSans*.ttf` هذا مشروع داخلي مخصص. الخطوط المرفقة (مرخصة بموجب Bitstream Vera License وتسمح بالاستخدام